Fun with *Toy* Computer

Part 1: Programming in Machine Language

Problem 1: Comments are helpful

Add helpful comments to the following machine language program and state what the program does.

```
PC: 07
00: 4865
01: 6c6c
02: 6f2c
03: 2077
04: 6f72
05: 6c64
06: 2100
07: 7000
08: b0c0
09: b0b0
0a: 0000
```

Problem 2: Changing bases

Write a machine language program that prompts "Input a number: ", inputs a signed value from the user and then outputs the value in binary, octal, denary and hexadecimal, each on a new line.

Example runs:

```
Input a number: 0xf
Binary: 1111
Octal: 17
Denary: 15
Hexadecimal: f

Input a number: -42
Binary: -101010
Octal: -52
Denary: -42
Hexadecimal: -2a
```

Problem 3: Multiplication

Notice that *Toy* does not have a machine instruction for *multiplication*. Write a machine code program that prompts the user for two positive numbers and then outputs the product of the two numbers. (Hint: treat multiplication as repeated addition.)

Example runs:

```
Input two positive numbers.
> 42
> 3
Product: 126

Input two positive numbers.
> 123
> 45
Product: 5535
```

Problem 4: Division

Write a machine language program that prompts the users for two integers A (you can assume the user will input a non negative number) and B (you can assume the user will input a positive number), and then outputs A div B and A mod B. (Hint: treat division as repeated subtraction.)

Example runs:

```
Input A: 39
Input B: 10
39 div 10: 3
39 mod 10: 9

Input A: 23
Input B: 7
23 div 7: 3
23 mod 7: 2
```

Problem 5: New instructions from old

Notice that *Toy* does not have machine instructions for the bitwise **not** and **or** operations (it only has **and**, **xor**, **left-shift** and **right-shift** bitwise operations).

(a) Write a machine language program that prompts the user for a (signed) integer and then outputs the the bit pattern of the number followed by the bit pattern of the result of the bitwise **not** operation applied to the number. (Hint: *subtraction* might be useful.)

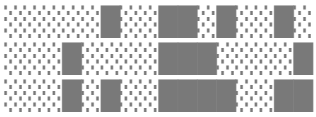
Example runs:

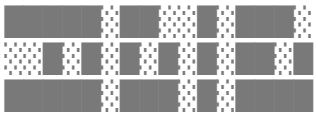
```
Input a number: 4242
```


```
Input a number: -42
```


(b) Write a machine language program that prompts the user for two (signed) numbers and then outputs the bit pattern of each number followed by the bit pattern of the result of the bitwise **or** operation applied to the numbers. (Hint: the bitwise **or** operation contains the bits that are set in *both* the operands *plus* the bits that are set in exactly one of the operands.)

Example runs:

```
Input two numbers.
> 1234
> 4321
```


```
Input two numbers.
> -1234
> 10925
```


Problem 6: Arithmetic sequences

Write a program in machine language that inputs three values (first two signed, last unsigned) from the user. The first number represents a first term, the second number represents a common difference and the third number represents how many terms are desired. The program then outputs that many terms of the arithmetic sequence thus described.

An example run:

```
First term: 16
Difference: -3
Terms: 10
16
13
10
7
4
1
-2
-5
-8
-11
```

Problem 7: The Fibonacci sequence

The Fibonacci sequence starts with the terms 1, 1, and then successive terms are obtained by adding the previous two terms, to generate: 1, 1, 2, 3, 5, 8, ...

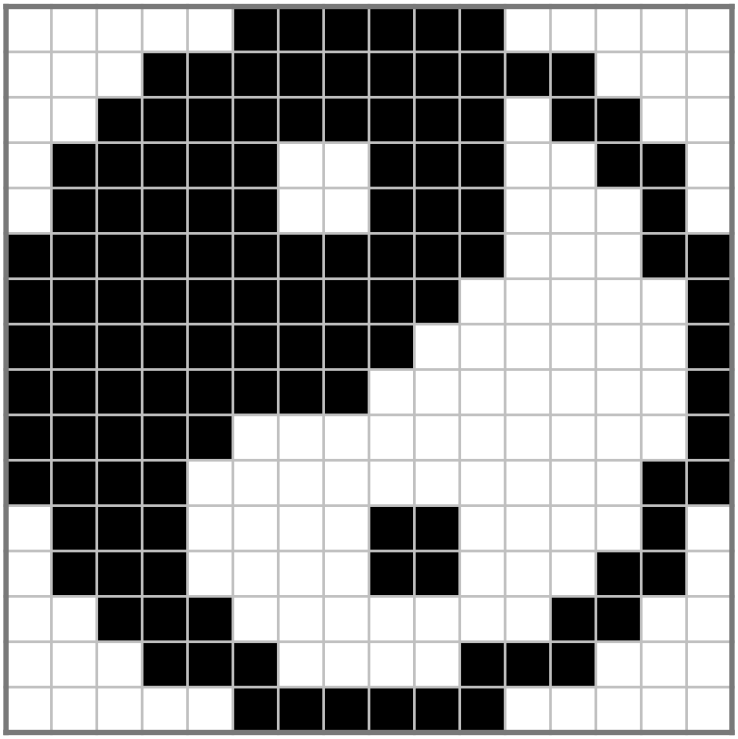
Write a machine language program that prompts the user for a number of terms and then outputs that number of terms from the Fibonacci sequence.

An example run:

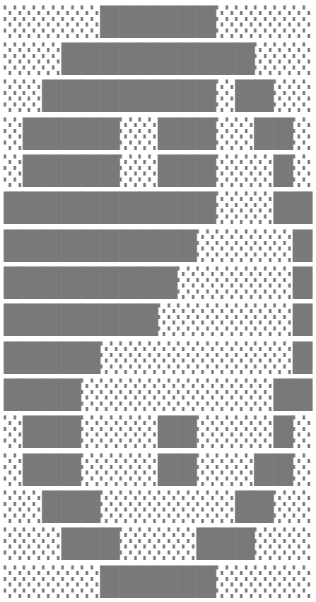
```
Terms: 15
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
```

Problem 8: Yin yang

Write a program in machine language that outputs the *yin yang* symbol to the terminal.



Expected output:

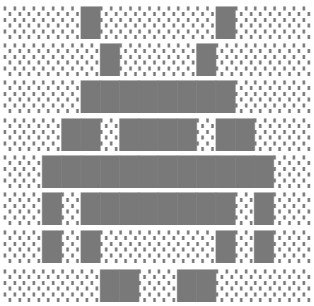


Problem 9: An image editor

Write a machine language program that inputs values from the user until the user inputs zero, and then generates an image from the bit patterns of the values.

An example run:

```
Input data.
0x0810
0x0420
0x0ff0
0x1bd8
0x3ffc
0x2ff4
0x2814
0x0660
0
```



Problem 10: Counting bits

Write a machine language program that asks the user to input signed values until the user inputs zero. For each value, the program outputs the binary representation of the value as it is stored in a register (i.e. the unsigned binary representation) and the number of bits that are set.

An example run:

```
Input values.
5
Data: 101
Bits: 2
42
Data: 101010
Bits: 3
-42
Data: 1111111111010110
Bits: 13
0
Bye!
```